

81 - Add or remove users from role

Към момента все още нямаме users in role, искаме да дадем възможност за редакция и обединим двата бутона:

Edit Role

Id: 71ef2a67-38d4-42e5-88c0-2847ce6f6ca0

RoleName: User

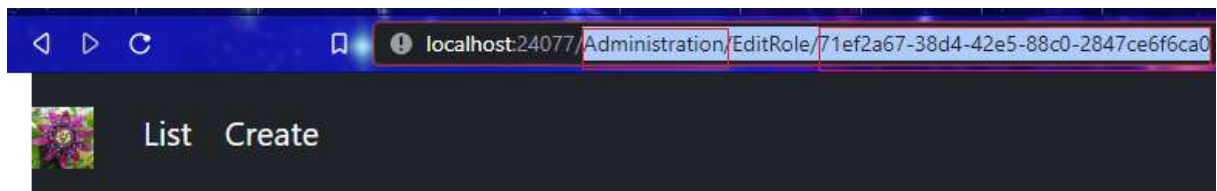
[Update](#) [Cancel](#)

Users in this role

None at the moment

[Add Users](#) [Remove Users](#)

```
<div class="card-footer">
  <a asp-action="EditUsersInRole" asp-controller="Administration"
    asp-route-roleId="@Model.Id" class="btn btn-primary"
    style="width:auto">Add or remove users</a>
</div>
```



Edit Role

Id

71ef2a67-38d4-42e5-88c0-2847ce6f6ca0

RoleName

User

Update

Cancel

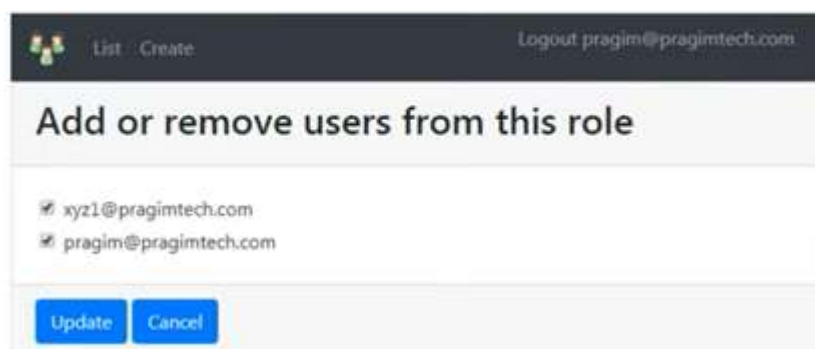
Users in this role

None at the moment

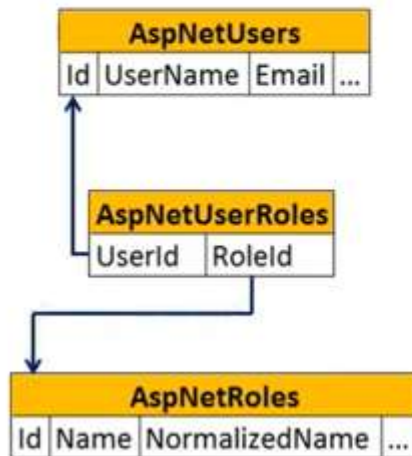
Add or remove users

localhost:24077/Administration/EditUsersInRole?roleId=71ef2a67-38d4-42e5-88c0-2847ce6f6ca0

Add or Remove Users from Role



AspNetUserRoles Identity Table



- Users are stored in **AspNetUsers** table
- Roles are stored in **AspNetRoles** table
- Many-to-Many relationship between **AspNetUsers** and **AspNetRoles** tables
- User and Role mapping data is stored in **AspNetUserRoles** table
- This table has just 2 columns : **UserId** & **RoleId**
- Both are **foreign keys**

Add or Remove Users from Role

AspNetUserRoles	
UserId	RoleId

Role 1-> n Users
RoleId -> UserId

not to duplicate
RoleId for every
user

```

public class UserRoleViewModel
{
    public string UserId { get; set; }
    public string RoleId { get; set; }
    public string UserName { get; set; }
    public bool IsSelected { get; set; }
}
  
```

move RoleId
out of
UserRoleViewModel
class

ViewBag.RoleId -> to pass it from the controller to the view.

```
UserRoleViewModel.cs  EditRole.cshtml  EditRoleViewMo
EmployeeManagement
namespace EmployeeManagement.Models
{
    0 references
    public class UserRoleViewModel
    {
        0 references
        public string UserId { get; set; }
        0 references
        public string UserName { get; set; }
        0 references
        public bool IsSelected { get; set; }
    }
}
```

```
AdministrationController.cs  EditRole.cshtml  EditRoleViewModel.cs  CreateRo
EmployeeManagement.Controllers.AdministrationController  EditUsersInF
[HttpGet]
0 references
public async Task<IActionResult> EditUsersInRole(string roleId)
{
    ViewBag.roleId = roleId;  store it to call it later

    var role = await roleManager.FindByIdAsync(roleId);

    if (role == null)
    {
        ViewBag.ErrorMessage = $"Role with Id = {roleId} cannot be found";
        return View("NotFound");
    }

    var model = new List<UserRoleViewModel>();

    foreach (var user in userManager.Users)
    {
        var userRoleViewModel = new UserRoleViewModel
        {
            UserId = user.Id,
            UserName = user.UserName
        };
    }
}
```

```

        if (await userManager.IsInRoleAsync(user, role.Name))
        {
            userRoleViewModel.IsSelected = true;
        }
        else
        {
            userRoleViewModel.IsSelected = false;
        }

        model.Add(userRoleViewModel);
    }

    return View(model);
}

```

```

[HttpGet]
public async Task<IActionResult> EditUsersInRole(string roleId)
{
    ViewBag.roleId = roleId;

    var role = await roleManager.FindByIdAsync(roleId);

    if (role == null)
    {
        ViewBag.ErrorMessage = $"Role with Id = {roleId} cannot be
found";
        return View("NotFound");
    }

    var model = new List<UserRoleViewModel>();

    foreach (var user in userManager.Users)
    {
        var userRoleViewModel = new UserRoleViewModel
        {
            UserId = user.Id,
            UserName = user.UserName
        };

        if (await userManager.IsInRoleAsync(user, role.Name))
        {
            userRoleViewModel.IsSelected = true;
        }
        else
        {
            userRoleViewModel.IsSelected = false;
        }

        model.Add(userRoleViewModel);
    }

    return View(model);
}

```


Administration folder -> EditUsersInRole.cshtml

```

EditUsersInRole.cshtml  UserRoleViewModel.cs  AdministrationController.cs  EditRole.cshtml  EditRoleV
@model List<UserRoleViewModel>

@{
    var roleId = ViewBag.roleId;
}

<form method="post">
    <div class="card">
        <div class="card-header">
            <h2>Add or remove users from this role</h2>
        </div>
        <div class="card-body">
            @for (int i = 0; i < Model.Count; i++)
            {
                <div class="form-check m-1">
                    <input type="hidden" asp-for="@Model[i].UserId" />
                    <input type="hidden" asp-for="@Model[i].UserName" />
                    <input asp-for="@Model[i].IsSelected" class="form-check-input" />
                    <label class="form-check-label" asp-for="@Model[i].IsSelected">
                        @Model[i].UserName
                    </label>
                </div>
            }
        </div>
        <div class="card-footer">
            <input type="submit" value="Update" class="btn btn-primary"
                style="width:auto" />
            <a asp-action="EditRole" asp-route-id="@roleId"
                class="btn btn-primary" style="width:auto">Cancel</a>
        </div>
    </div>
</form>

```

```
@model List<UserRoleViewModel>
```

```
@{
    var roleId = ViewBag.roleId;
}
```

```
<form method="post">
    <div class="card">
        <div class="card-header">
            <h2>Add or remove users from this role</h2>
        </div>
        <div class="card-body">
            @for (int i = 0; i < Model.Count; i++)
            {
                <div class="form-check m-1">
                    <input type="hidden" asp-for="@Model[i].UserId" />
                    <input type="hidden" asp-for="@Model[i].UserName" />
                    <input asp-for="@Model[i].IsSelected" class="form-check-
input" />
                    <label class="form-check-label" asp-
for="@Model[i].IsSelected">
                        @Model[i].UserName
                    </label>
                </div>
            }
        </div>
    </div>
</form>
```

```

    }
  </div>
  <div class="card-footer">
    <input type="submit" value="Update" class="btn btn-primary"
      style="width:auto" />
    <a asp-action="EditRole" asp-route-id="@roleId"
      class="btn btn-primary" style="width:auto">Cancel</a>
  </div>
</div>
</form>

```

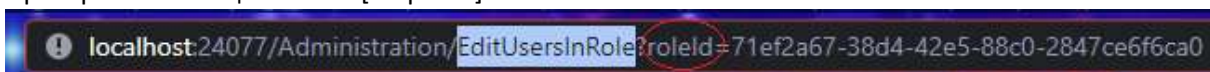
Към момента, никой от потребителите не е в тази role, затова никой check-box не е маркиран. Ако маркираме и дадем Update, ще трябва да се запази в БД. Освен при маркиране на тоглера, искаме при кликане върху самият надпис на потребителя да се селектира и асоциирания към него тоглер. Това се изпълнява с

```

<input asp-for="@Model[i].IsSelected" class="form-check-input" />
<label class="form-check-label" asp-for="@Model[i].IsSelected">
  @Model[i].UserName
</label>

```

При Update все още нямаме [HttpPost] EditUsersInRole action in AdministrationController -> 404



Добавяме го, според променливите, които сме получили: лист + roleId от URL.

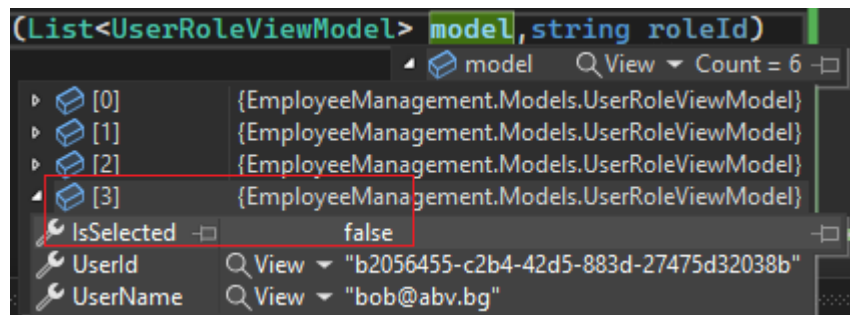
```

151 [HttpPost]
152 // references
153 public async Task<ActionResult> EditUsersInRole(List<UserRoleViewModel> model, string roleId)
154 {
155     return View(model);
156 }

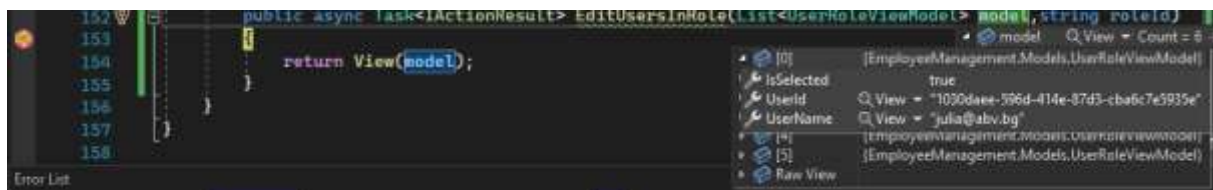
```

Добавяме Breakpoint -> Debug mode, селектирам 1,2,3 -> Update:

- ☒ julia@abv.bg
- ☒ aurora@abv.bg
- ☒ sara@abv.bg
- ☐ bob@abv.bg
- ☐ david@abv.bg
- ☐ tim@abv.bg

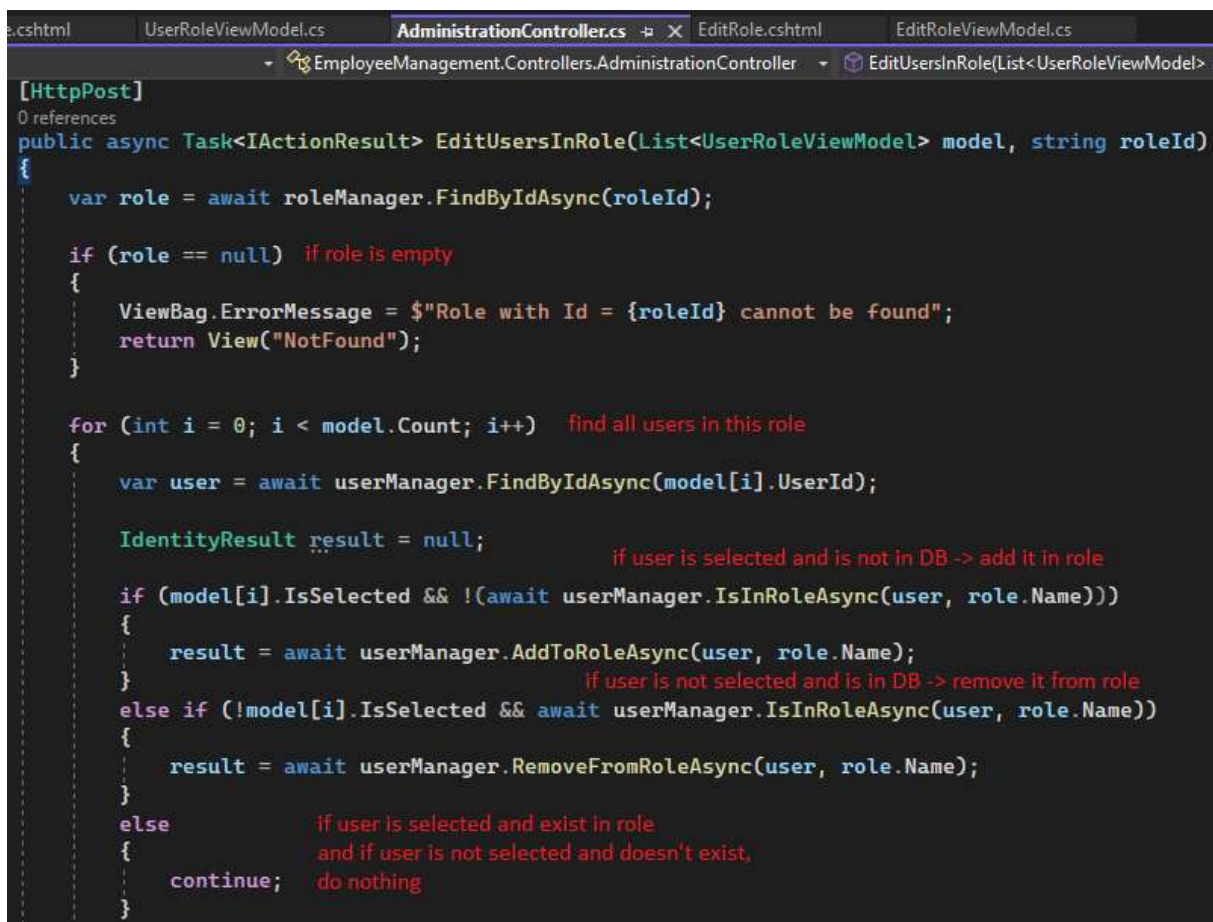


Breakpoint is hit: - виждаме, че model съдържа 6 стойности, като за първата IsSelected = true, (маркирана е), по същият начин като 2,3 докато 4,5,6 не са избрани, съответно false.



Другите данни, които се пренасят, са за UserID, UserName, които са `<input type="hidden">`

Спираме дебъгването.




```

        if (result.Succeeded)
        {
            if (i < (model.Count - 1)) there are more model objects to proceed
                continue;
            else end of the loop
                return RedirectToAction("EditRole", new { Id = roleId });
        }

        if model parameter is empty, there are any userRole model objects
        return RedirectToAction("EditRole", new { Id = roleId });
    }
}

```

```

using EmployeeManagement.Models;
using EmployeeManagement.ViewModels;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Identity;
using Microsoft.AspNetCore.Mvc;
using System.Collections.Generic;
using System.Threading.Tasks;

namespace EmployeeManagement.Controllers
{
    public class AdministrationController : Controller
    {
        private readonly RoleManager<IdentityRole> roleManager;
        private readonly UserManager<ApplicationUser> userManager;

        public AdministrationController(RoleManager<IdentityRole> roleManager,
                                         UserManager<ApplicationUser> userManager)
        {
            this.roleManager = roleManager;
            this.userManager = userManager;
        }

        [HttpGet]
        public IActionResult CreateRole()
        {
            return View();
        }

        [HttpPost]
        public async Task<IActionResult> CreateRoleAsync(CreateRoleViewModel
model)
        {
            if (ModelState.IsValid)
            {
                IdentityRole identityRole = new IdentityRole
                {
                    Name = model.RoleName
                };
                IdentityResult result = await
roleManager.CreateAsync(identityRole);

                if (result.Succeeded)
                {
                    return RedirectToAction("listroles", "administration");
                }
                foreach (IdentityError error in result.Errors)
                {
                    ModelState.AddModelError("", error.Description);
                }
            }
        }
    }
}

```

```

        }
    }
    return View(model);
}
[HttpGet]
public IActionResult ListRoles()
{
    var roles = roleManager.Roles;
    return View(roles);
}
// Role ID is passed from the URL to the action
[HttpGet]
public async Task<IActionResult> EditRole(string id)
{
    var role = await roleManager.FindByIdAsync(id);

    if (role == null)
    {
        ViewBag.ErrorMessage = $"Role with Id = {id} cannot be found";
        return View("NotFound");
    }

    var model = new EditRoleViewModel
    {
        Id = role.Id,
        RoleName = role.Name
    };

    foreach (var user in userManager.Users)
    {
        if (await userManager.IsInRoleAsync(user, role.Name))
        {
            model.Users.Add(user.UserName);
        }
    }

    return View(model);
}

[HttpPost]
public async Task<IActionResult> EditRole(EditRoleViewModel model)
{
    var role = await roleManager.FindByIdAsync(model.Id);

    if (role == null)
    {
        ViewBag.ErrorMessage = $"Role with Id = {model.Id} cannot be
found";
        return View("NotFound");
    }
    else
    {
        role.Name = model.RoleName;

        var result = await roleManager.UpdateAsync(role);

        if (result.Succeeded)
        {
            return RedirectToAction("ListRoles");
        }

        foreach (var error in result.Errors)
        {

```

```

        ModelState.AddModelError("", error.Description);
    }

    return View(model);
}

[HttpGet]
public async Task<IActionResult> EditUsersInRole(string roleId)
{
    ViewBag.roleId = roleId;

    var role = await roleManager.FindByIdAsync(roleId);

    if (role == null)
    {
        ViewBag.ErrorMessage = $"Role with Id = {roleId} cannot be
found";
        return View("NotFound");
    }

    var model = new List<UserRoleViewModel>();

    foreach (var user in userManager.Users)
    {
        var userRoleViewModel = new UserRoleViewModel
        {
            UserId = user.Id,
            UserName = user.UserName
        };

        if (await userManager.IsInRoleAsync(user, role.Name))
        {
            userRoleViewModel.IsSelected = true;
        }
        else
        {
            userRoleViewModel.IsSelected = false;
        }

        model.Add(userRoleViewModel);
    }

    return View(model);
}

[HttpPost]
public async Task<IActionResult> EditUsersInRole(List<UserRoleViewModel>
model, string roleId)
{
    var role = await roleManager.FindByIdAsync(roleId);

    if (role == null)
    {
        ViewBag.ErrorMessage = $"Role with Id = {roleId} cannot be
found";
        return View("NotFound");
    }

    for (int i = 0; i < model.Count; i++)
    {
        var user = await userManager.FindByIdAsync(model[i].UserId);

        IdentityResult result = null;
    }
}

```

```

        if (model[i].IsSelected && !(await
userManager.IsInRoleAsync(user, role.Name)))
        {
            result = await userManager.AddToRoleAsync(user, role.Name);
        }
        else if (!model[i].IsSelected && await
userManager.IsInRoleAsync(user, role.Name))
        {
            result = await userManager.RemoveFromRoleAsync(user,
role.Name);
        }
        else
        {
            continue;
        }

        if (result.Succeeded)
        {
            if (i < (model.Count - 1))
                continue;
            else
                return RedirectToAction("EditRole", new { Id = roleId });
        }
    }

    return RedirectToAction("EditRole", new { Id = roleId });
}
}
}

```

Правим проверки:

- ☐ julia@abv.bg
- ☐ aurora@abv.bg
- ☒ sara@abv.bg
- ☒ bob@abv.bg
- ☐ david@abv.bg
- ☐ tim@abv.bg

	UserId	RoleId
	1030daee-596d-414e-87d3-cba6c7e5935e	71ef2a67-38d4-42e5-88c0-2847ce6f6ca0
	1f9ea9da-40d8-4413-a222-9fd26303740c	71ef2a67-38d4-42e5-88c0-2847ce6f6ca0
	87cabcf7-6319-4672-a7c8-db5a048fa5f7	7a53c902-d9f9-4026-b521-26b8485f1611
	b2056455-c2b4-42d5-883d-27475d32038b	7a53c902-d9f9-4026-b521-26b8485f1611
	NULL	NULL

Edit Role

Id: 71ef2a67-38d4-42e5-88c0-2847ce6f6ca0

RoleName: User

Users in this role

julia@abv.bg
aurora@abv.bg

Edit Role

Id: 7a53c902-d9f9-4026-b521-26b8485f1611

RoleName: Admin

Users in this role

sara@abv.bg
bob@abv.bg